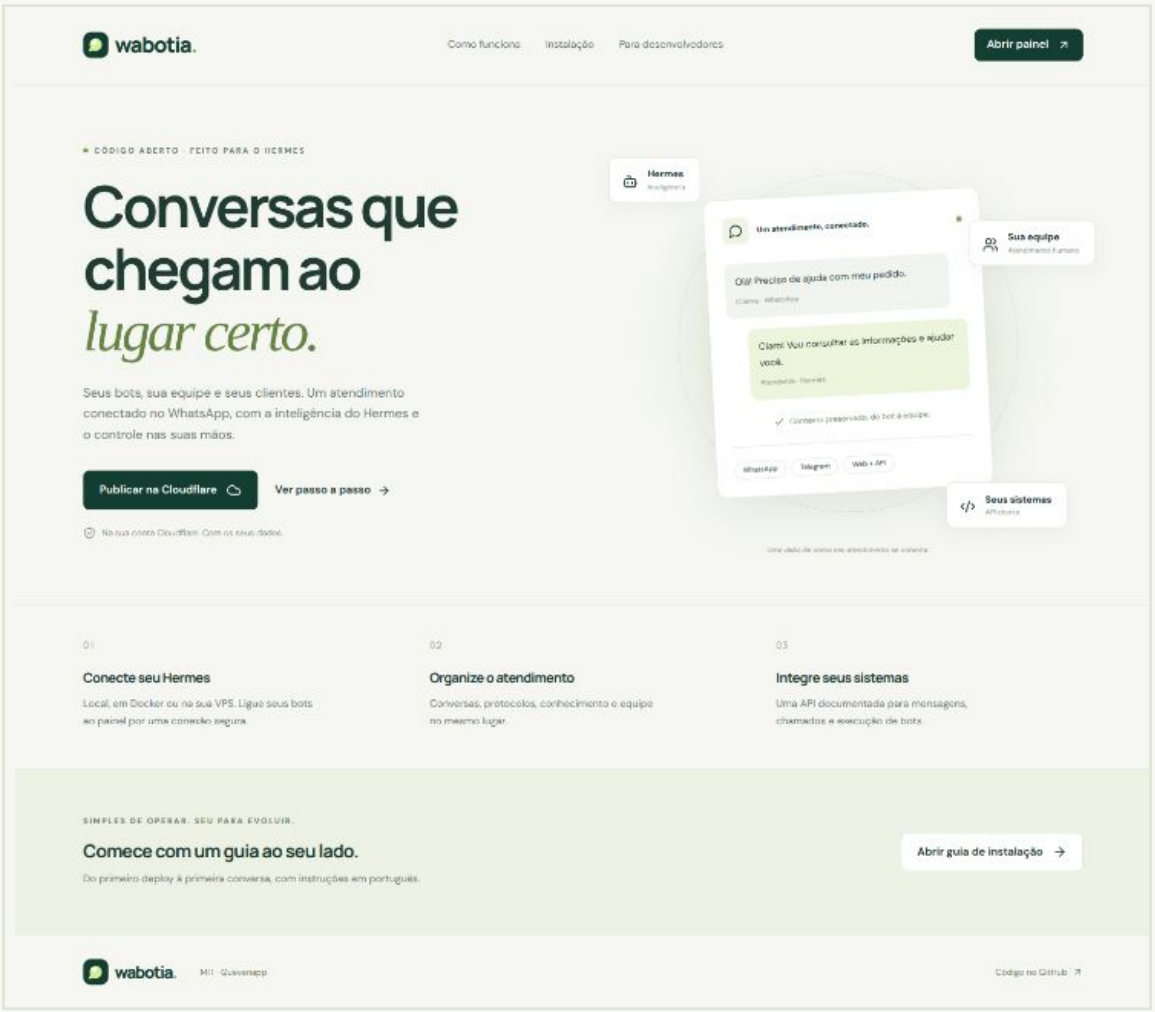
Seu atendimento. Bem conectado.

Guia para instalar na Cloudflare, conectar o Hermes e preparar sua equipe.



Página inicial do Wabotia. As ilustrações e conversas de exemplo são fictícias.

Começar em wabotia.queven.app/install

Edição de 05/11/2026. Leia na ordem; escolha apenas um modo de WhatsApp por número. O painel fica na Cloudflare. Hermes e QR continuam no computador ou servidor que você escolher.

Primeiro, vincule suas contas

Antes do botão de deploy do Wabotia, conecte a Cloudflare ao GitHub. Se sua conta já aparece em **Git account** na Cloudflare, continue para os requisitos nas próximas páginas.

1. Entre na Cloudflare, em Workers & Pages e selecione a conta onde vai instalar o Wabotia.

2. Clique **Create application / Criar aplicativo** e escolha **GitHub**. Em algumas telas, a opção fica em **Import a repository / Importar repositório**.

3. Use **Connect GitHub / Conectar GitHub** ou **+ Add account / Adicionar conta**. No GitHub, escolha sua conta ou organização e autorize **Cloudflare Workers and Pages**. Confira as permissões solicitadas.

4. De volta à Cloudflare, confira sua conta em **Git account**. A vinculação terminou: volte ao site Wabotia. Confira os requisitos das próximas páginas antes de abrir o botão específico do Wabotia.

A tela fechou depois de autorizar? Isso pode acontecer na primeira vinculação pelo botão. Volte ao site Wabotia e clique novamente em **Deploy to Cloudflare**. Selecione a conta já vinculada para continuar. Autorizar o GitHub, sozinho, ainda não instala o Wabotia.

Antes de gerar chaves e clicar em Deploy, prepare os serviços das páginas 3 a 5. Elas explicam o que cada serviço faz e como confirmar que R2, D1 e Zero Trust estão disponíveis.

Para que serve a Cloudflare?

Ela mantém o Wabotia disponível na internet e guarda seus dados. Cada serviço cuida de uma parte. O Hermes continua executando os bots na máquina ou VPS escolhida.

Serviço	Função no seu atendimento
Workers	Executa o painel web, a API e as regras do atendimento. O painel continua acessível com seu computador desligado.
D1	Banco de dados: guarda usuários, configurações, conversas, protocolos e registros.
R2	Armazena documentos, imagens, áudios e anexos. Um bucket é a área onde esses arquivos ficam guardados. Use bucket privado.
Durable Objects e alarmes	Organizam o processamento por perfil e retomam tarefas pendentes, mantendo a ordem e evitando operações duplicadas.
Zero Trust	Área da Cloudflare para administrar a proteção do acesso ao Hermes. Exige ativar um plano, como Zero Trust Free.
Access	Confere se a chamada ao Hermes vem do Wabotia autorizado. Usa aplicações, políticas Service Auth e um par Client ID/Secret.
Tunnel e cloudflared	Ligam a Cloudflare à máquina do Hermes sem abrir portas no roteador. cloudflared é o programa instalado nessa máquina.
DNS	Associa nomes como api-hermes.seudominio.com ao túnel correto. O domínio precisa estar ativo na Cloudflare.
Workers Builds	Busca o código no GitHub, prepara e publica o Wabotia pelo botão de deploy. É separado do GitHub Actions.
Workers Secrets	Guardam MASTER_KEY e SETUP_TOKEN no ambiente protegido do Worker. Não publique esses valores no código.

Exemplo: a mensagem chega ao Worker; o registro vai para o D1 e o anexo para o R2. Durable Objects organizam o trabalho. Para executar um bot, o Worker passa por Access + Tunnel até o Hermes.

O plano do site/domínio, Workers, a assinatura R2 e Zero Trust são configurações distintas. Ativar um deles não ativa automaticamente os demais. D1 e Durable Objects seguem o plano Workers.

Ative R2 e confira D1

R2: adesão necessária, mesmo com franquia gratuita

1. Na conta Cloudflare da instalação, abra **Armazenamento e bancos de dados > R2 > Visão geral**.

2. Se aparecer a tela de adesão, conclua a ativação da assinatura R2. Preencha os dados de cobrança solicitados e confira termos e valores antes de confirmar. A franquia gratuita não dispensa essa adesão.

3. Confirme que aparece a lista de buckets ou **Criar bucket**. Você pode preparar um bucket privado chamado wabotia-media e selecioná-lo no deploy. Se o bucket dessa instalação já existe, use-o.

R2 inclui uma franquia mensal gratuita e pode cobrar pelo uso excedente. Confira as condições atuais; gratuito não significa armazenamento ilimitado.

D1: banco incluído no plano Workers

4. Abra **Armazenamento e bancos de dados > Banco de dados SQL D1**. Confira a lista e o botão **Criar banco de dados**.

5. O D1 já faz parte de **Workers Free e Paid**. Não há uma assinatura D1 separada equivalente à adesão do R2. Limites e eventuais cobranças seguem o plano Workers.

6. O deploy cria ou conecta o banco exclusivo. Não é necessário criar um banco de teste antes. Se já instalou, confira **Worker > Bindings > DB** e preserve o banco existente.

Confira o plano Workers

Workers Paid é a referência de produção do Wabotia. Confira o plano e as condições na área de assinatura do Workers. Zero Trust Free e o plano do domínio não alteram essa assinatura.

O deploy cria as ligações **DB** e **MEDIA** entre Worker, D1 e R2. Você não precisa gerar chaves D1 ou credenciais S3/R2 para preencher no Wabotia. Durable Objects e alarmes também são configurados pelo deploy.

Ativação oficial do R2 · Preços R2 · Planos D1

Zero Trust e requisitos finais

1. Para conectar Hermes privado, abra **Zero Trust > Controles de acesso > Aplicativos**. Se aparecer a apresentação, clique **Começar** e conclua a ativação do plano. Zero Trust Free atende esta configuração inicial.

2. Confira termos e eventuais informações de cobrança, mesmo escolhendo gratuito. Se pedir nome de equipe, escolha um disponível.

3. Confirme acesso a **Aplicativos** e **Credenciais de serviço**. Se já ativou, não repita a adesão. O Service Token, as políticas Access e o túnel serão criados depois, nas páginas 15 a 19.

Antes de clicar no botão de deploy

Requisito	Como saber que está pronto
GitHub vinculado	A conta aparece em Git account na Cloudflare.
R2 ativado	Lista de buckets ou Criar bucket disponível.
D1 e Workers conferidos	Lista de bancos/Criar banco de dados acessível; plano Workers escolhido para o uso.
Zero Trust ativado	Aplicativos e Credenciais de serviço acessíveis.
Domínio e Hermes	Domínio ativo para os hostnames do túnel; Hermes com modelo/provedor configurados.

Agora gere e guarde as duas chaves

No gerador do site, clique **Gerar minhas duas chaves**. Guarde ambas no gerenciador de senhas ou baixe o arquivo privado. MASTER_KEY protege as credenciais; SETUP_TOKEN autoriza a primeira conta. A senha de login será escolhida depois.

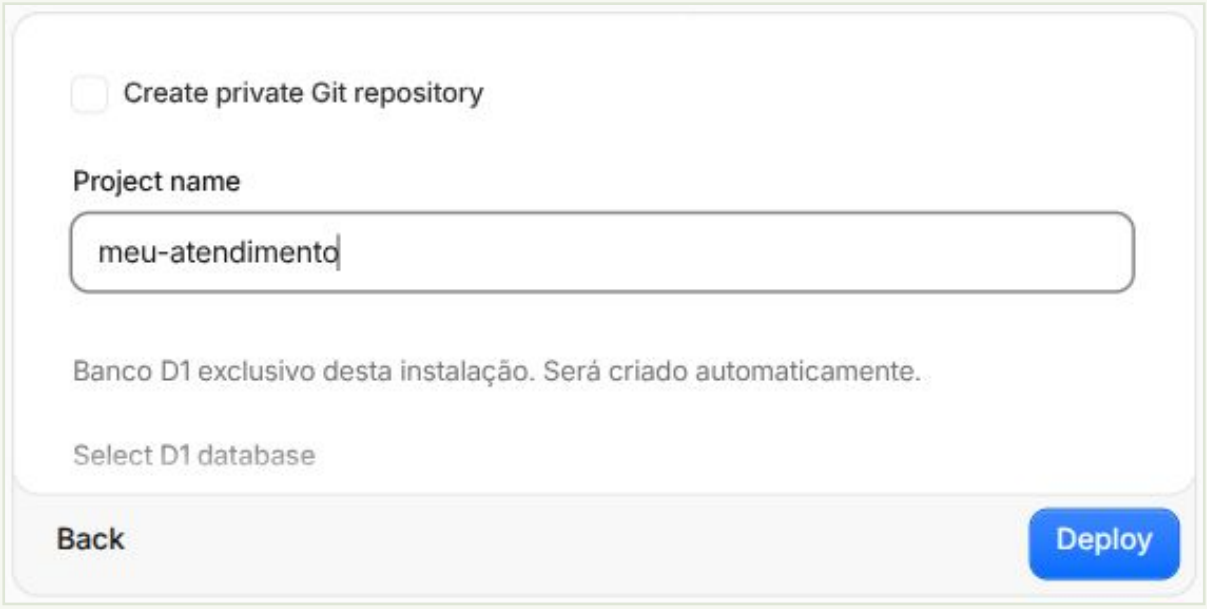
As chaves somem da tela ao atualizar/fechar a página. Se já instalou, preserve as originais. Pelo instalador de terminal, as chaves são geradas e guardadas automaticamente.

Continue pelo **botão nas páginas 6 e 7** ou pelo **terminal na página 8**. Para testar os canais, tenha um número de teste e, se usar Telegram, um bot dedicado com grupo privado de tópicos.

Publicar pelo botão

1. Com as contas vinculadas e os requisitos conferidos, volte a wabotia.queven.app/install. Gere e guarde as duas chaves. Clique **Deploy to Cloudflare**: abre outra aba.

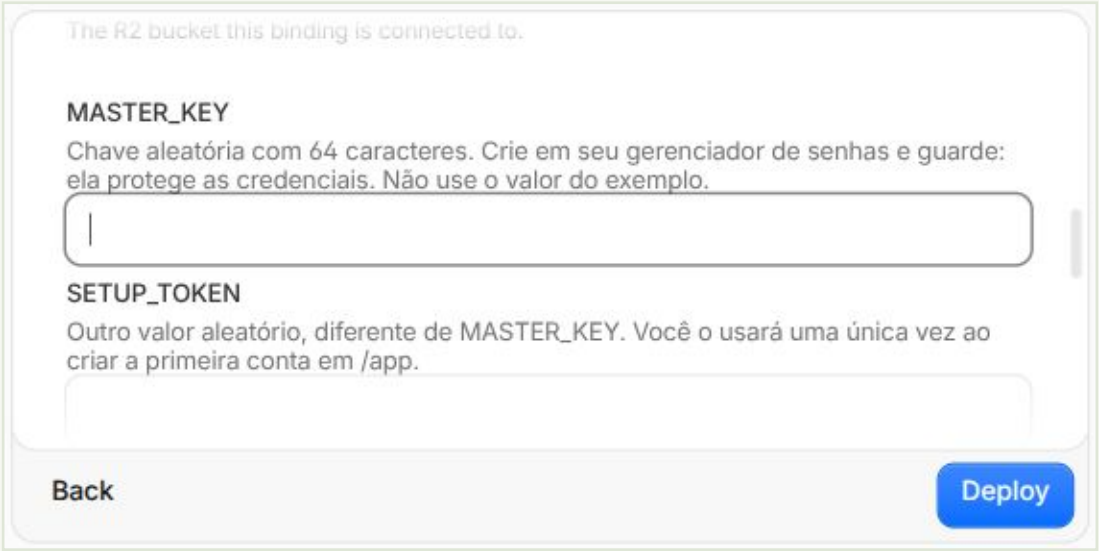
2. Confira a origem **Quevenapp/Wabotia**. Escolha a conta GitHub já vinculada em **Git account** e um nome de projeto. Será criada uma cópia do código na conta escolhida.



Tela real da Cloudflare, com nome fictício. Role dentro do formulário para ver os próximos campos.

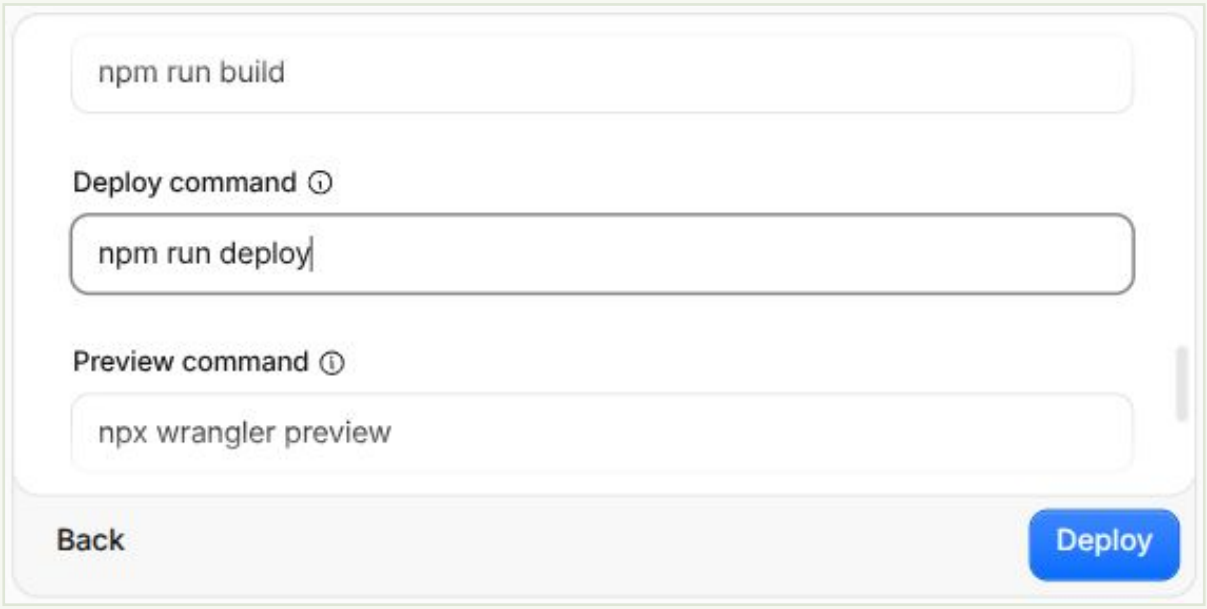
3. Use um banco D1 e um bucket R2 exclusivos desta instalação. Mantenha o bucket privado. Para uma segunda instalação, escolha recursos diferentes.

4. Volte à aba do gerador. Use **Copiar MASTER_KEY** e cole no campo MASTER_KEY da Cloudflare. Repita com **Copiar SETUP_TOKEN** no campo SETUP_TOKEN. Guarde as duas antes de fechar o gerador.



Os dois campos ficam vazios nesta referência. Nunca publique uma captura com suas chaves.

Concluir o deploy



Mantenha os comandos detectados pelo repositório. O formulário foi conferido no Chrome.

1. Em **Build command**, use **npm run build**. Em **Deploy command**, use **npm run deploy**: o comando publica o Worker e aplica as migrações D1.

2. Desative **Enable Preview builds** até preparar recursos isolados para testes. Nunca reutilize o banco de produção para experimentar mudanças.

3. Clique **Deploy** e aguarde a indicação de sucesso. Guarde a URL que termina em **workers.dev**.

4. No Worker, abra **Settings > Variables and Secrets**. Confira a existência de MASTER_KEY e SETUP_TOKEN. Se não foram solicitados, adicione ambos como **Secret**.

5. Abra a URL da **sua instalação**, acrescente **/app** e siga para a página 9.

GitHub Actions: o repositório original mantém Actions desativado. Na sua cópia, confira Settings > Actions > General > Disable actions. O botão usa Cloudflare Builds, que é outro serviço.

O botão foi validado em uma instalação real. Se falhar ao criar R2, crie o bucket privado em R2 e selecione-o no formulário. Se a URL não abrir após sucesso, habilite workers.dev em Settings > Domains & Routes do Worker.

Alternativa: instalador

Este caminho gera as chaves automaticamente e foi validado com recursos reais na Cloudflare. Use-o se preferir acompanhar a instalação pelo terminal.

1. Instale **Node.js LTS** (22.13 ou superior; prefira 24 LTS) e **Git** pelos sites oficiais.

2. Abra Terminal no Linux/macOS, Ubuntu no Windows/WSL2, ou PowerShell com Node e Git instalados. Cole uma linha de cada vez e pressione Enter.

```
git clone https://github.com/Quevenapp/Wabotia.git
cd Wabotia
npm ci
npx wrangler login
npm run setup
```

3. No navegador aberto pelo login, escolha a conta Cloudflare correta. Volte ao terminal após a confirmação.

4. Aguarde as cinco etapas: acesso, recursos, banco, secrets e endereço final. Não feche o processo antes da conclusão.

5. Na pasta Wabotia, abra **.secrets.production.local.json** em um editor. Arquivos iniciados por ponto podem estar ocultos. Guarde esse arquivo protegido e copie SETUP_TOKEN somente quando o painel pedir.

6. Abra a URL do Worker, com **/app** ao final. Siga para a página 9.

Teste e domínio próprio

```
npm run setup -- --environment staging
npm run setup -- --domain painel.seudominio.com
```

Use a primeira linha para um ambiente de teste separado; a segunda, se quiser domínio próprio na mesma conta Cloudflare. Não execute o instalador sobre recursos existentes com uma nova MASTER_KEY. Preserve os arquivos .local.json da implantação.

Sua primeira conta

Abra **SUA_URL/app**. A primeira conta é de administrador. As próximas pessoas serão cadastradas dentro do painel.



Tela real de primeira instalação. Os campos foram mantidos vazios.

1. Preencha nome, e-mail e senha com pelo menos 12 caracteres. Cole o **SETUP_TOKEN** gerado ou definido no deploy.

2. Clique **Criar conta e entrar**. Depois clique **Criar perfil de atendimento** ou no **+** ao lado do seletor de perfil. Dê um nome, como Atendimento da loja.

3. Mantenha o atendimento pausado. Em **Pessoas**, crie uma conta por atendente e selecione os perfis permitidos.

Hermes: por onde começar

Se o Wabotia já está instalado, mantenha as chaves originais. Agora você vai ligar as APIs do Hermes, proteger a conexão na Cloudflare e cadastrar o perfil no painel.

1. Escolha onde está o Hermes: **computador nativo** (páginas 11 e 12) ou **VPS Ubuntu com Docker** (páginas 13 e 14).

2. Teste as APIs localmente. Só continue depois dos dois resultados **OK**.

3. Você pode usar o **botão de deploy pelo painel, páginas 30 a 32**, ou configurar Zero Trust, Access e Tunnel manualmente nas páginas 15 a 19.

4. Cadastre a conexão no Wabotia, página 20. A página 21 explica como religar serviços.

Cada chave tem um lugar

Credencial	Quem gera / onde usar
MASTER_KEY e SETUP_TOKEN	Gerador/instalador Wabotia. Secrets do Worker, já guardados na instalação. Preserve os valores originais.
Token administrativo Hermes	Assistente prepare-hermes.py. Campo administrativo da conexão no Wabotia.
API_SERVER_KEY de wabotia	Assistente ou Hermes. Campo da chave do perfil na conexão.
Access Client ID e Client Secret	Cloudflare. Preencha o par nos campos Cloudflare Access da conexão.
Token do túnel	Cloudflare. O assistente o guarda criptografado para gerar o instalador privado do cloudflared. No modo manual, use apenas na máquina.

As chaves Hermes geradas ficam em **~/.config/wabotia/hermes/chaves.local.txt**. Abra em um editor, copie apenas os valores e guarde o arquivo privado. No Linux, Ctrl+H mostra pastas ocultas. Não publique chaves nem capturas que as revelem.

Comandos completos e referências oficiais

Local: habilite a API

Abra Terminal no Ubuntu/Linux/macOS, ou Ubuntu no Windows/WSL2, com o mesmo usuário que instalou o Hermes. Confirme que **hermes --help** funciona. Não use sudo nos comandos Hermes.

1. Se não tem uma cópia do código Wabotia, baixe-a. Se já tem, abra e atualize a pasta existente.

```
git clone https://github.com/Quevenapp/Wabotia.git
cd Wabotia
```

2. Execute a preparação abaixo. Troque **seudominio.com** pelo domínio ativo na sua Cloudflare. O perfil wabotia é criado somente se ainda não existir.

```
python3 scripts/prepare-hermes.py \
  --profile wabotia --create-profile \
  --admin-hostname hermes-local.seudominio.com \
  --api-hostname api-hermes-local.seudominio.com
```

3. Aguarde as quatro etapas do assistente. Ele faz backup privado, preserva chaves existentes e cria as que faltam.

4. Abra o perfil **wabotia** no Hermes e confira modelo, provedor e autenticação. Se necessário, use o seletor nativo:

```
hermes -p wabotia model
```

API desligada: a preparação configura gateway.api_server.enabled=true, host=127.0.0.1 e porta 8642. Criar a chave sozinho não inicia o processo. A próxima página liga os serviços.

O gateway é compartilhado entre perfis. A chave default, necessária para o listener, permanece no Hermes; o Wabotia recebe somente a chave do perfil escolhido. Canais e dados existentes são preservados. O assistente recusa perfis pausados e gateway.multiplex_profiles=false.

Local: ligue e teste

No Ubuntu/Linux com gateway instalado como serviço de usuário, cole uma linha de cada vez. A barra \ no fim da linha indica continuação do mesmo comando.

```
mkdir -p ~/.config/systemd/user
install -m 600 \
  ~/.config/wabotia/hermes/hermes-wabotia-admin.service \
  ~/.config/systemd/user/hermes-wabotia-admin.service
systemctl --user daemon-reload
systemctl --user enable --now hermes-wabotia-admin.service
systemctl --user restart hermes-gateway.service
```

Reinicie fora de um atendimento em andamento: o gateway pausa brevemente seus canais. Se aparecer **Unit not found**, use o gerenciador pelo qual seu Hermes já inicia. Não abra um segundo gateway. Sem gateway instalado, consulte **hermes gateway install --help**.

Confira o resultado antes de abrir o túnel

```
python3 scripts/prepare-hermes.py --check

OK · Descoberta: autenticado e recusando acesso sem chave.
OK · API de bots: autenticado e recusando acesso sem chave.
```

Esse teste confirma a descoberta, idempotência durável e autenticação. Não executa o modelo nem envia mensagens. Só prossiga quando as duas linhas estiverem OK.

Execução manual, Mac e persistência

No macOS ou sem systemd, mantenha este comando em outro terminal. Reinicie o gateway pelo método já usado:

```
python3 scripts/prepare-hermes.py --serve-admin
```

Fechar o terminal encerra esse backend. No Mac, use launchd para início automático. No Ubuntu, para manter os servicos de usuário após sair da sessão:

```
sudo loginctl enable-linger "$USER"
```

No Windows/WSL2, systemd precisa estar habilitado e o WSL deve permanecer ligado. Suspender ou desligar o computador interrompe os bots.

VPS Docker: prepare

Entre por SSH ou pelo terminal da Hostinger. Este roteiro usa a imagem oficial atual, usuário **hermes**, volume **/opt/data** e rede **host** no Ubuntu. Confira sua implantação antes de aplicar.

```
docker ps --format 'table {{.Names}}\t{{.Status}}'
docker inspect -f '{{.HostConfig.NetworkMode}}' hermes
```

O exemplo usa contêiner chamado hermes. Substitua esse nome em todos os comandos se o seu for diferente. O segundo resultado precisa ser **host**. Em rede bridge, adapte a rede privada conforme o manual completo; o localhost do host não é o contêiner.

1. Faça backup do volume e do Compose existentes. Não use docker compose down -v.

2. Na pasta do código Wabotia da VPS, copie e execute o assistente dentro do contêiner:

```
docker cp scripts/prepare-hermes.py \
  hermes:/tmp/wabotia-prepare-hermes.py
docker exec --user hermes hermes python3 \
  /tmp/wabotia-prepare-hermes.py \
  --output /opt/data/wabotia-connection \
  --profile wabotia --create-profile \
  --admin-hostname hermes-vps.seudominio.com \
  --api-hostname api-hermes-vps.seudominio.com
docker exec --user hermes hermes cp \
  /tmp/wabotia-prepare-hermes.py \
  /opt/data/wabotia-connection/prepare-hermes.py
```

3. Fora de um atendimento, reinicie o contêiner existente para carregar a API:

```
docker restart hermes
```

Confira modelo/provedor no perfil novo. Se a imagem não tiver o usuário, Python ou API compatível, revise a versão; não contorne as permissões executando tudo como root. A execução na sua VPS precisa ser validada.

VPS Docker: mantenha ligado

Na VPS, crie **/etc/systemd/system/hermes-wabotia-admin.service** com sudo nano. O serviço abaixo mantém só o backend administrativo dentro do contêiner existente, sem iniciar outro gateway.

```
[Unit]
Description=Hermes administrativo para Wabotia
After=docker.service
Requires=docker.service

[Service]
Type=simple
ExecStart=/usr/bin/docker exec --user hermes hermes python3 \
  /opt/data/wabotia-connection/prepare-hermes.py \
  --output /opt/data/wabotia-connection --serve-admin
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
```

Confira **command -v docker** e ajuste o caminho se necessário. A porta 9119 precisa estar livre. Caso contrário, prepare com --admin-port 9120 e use 9120 também no túnel.

```
sudo systemctl daemon-reload
sudo systemctl enable --now hermes-wabotia-admin.service
docker exec --user hermes hermes python3 \
  /opt/data/wabotia-connection/prepare-hermes.py \
  --output /opt/data/wabotia-connection --check
```

Espere os dois OK da página 12. Copie as chaves para um arquivo privado no host com os comandos do manual completo. Dentro do contêiner elas estão em **/opt/data/wabotia-connection/chaves.local.txt**.

Instale cloudflared no host Ubuntu. Na rede host, ele alcança as APIs em 127.0.0.1. Use o túnel e os hostnames **da VPS**. Preserve o volume e o reinício automático do contêiner ao atualizar o Hermes.

Cloudflare: Zero Trust

Se ativou Zero Trust nos requisitos iniciais, confira o acesso a Aplicativos e avance à próxima página. A referência abaixo orienta quem ainda precisa concluir a adesão. Use a conta com seu domínio ativo.



Tela real de ativação do Zero Trust. Confira os valores e termos vigentes antes de concluir.

1. Se aparecer a tela de boas-vindas, clique **Começar**. Selecione o plano adequado. Zero Trust gratuito, quando disponível, atende esta conexão inicial.

2. Confira preço, eventuais dados de cobrança e termos antes de confirmar a ativação. O plano Zero Trust é separado do Workers Paid.

3. Se pedir nome de equipe, escolha um disponível, como minhaempresa-wabotia. O endereço nome.cloudflareaccess.com não é a URL do Hermes.

4. Volte a **Controles de acesso > Aplicativos**. Continue quando conseguir acessar a lista e adicionar aplicações.

A ativação precisa ser concluída pelo titular da conta quando houver aceite de termos ou cobrança. Ter um túnel já criado não significa que o Access esteja habilitado.

Access: proteja as duas URLs

1. Abra **Controles de acesso > Credenciais de serviço > Service Tokens**. Em telas antigas: Access > Service Auth > Service Tokens.

2. Clique **Criar token de serviço**. Nome: wabotia-hermes-local. Escolha validade e anote a data de renovação.

3. A Cloudflare gera **Client ID** e **Client Secret**. Guarde os dois agora; o segredo pode aparecer apenas uma vez.

4. Em **Aplicativos > Adicionar aplicativo > Self-hosted**, preencha conforme a referência abaixo.

Referência de preenchimento: aplicação administrativa

Campo	Preencher
Nome	Wabotia - Hermes administrativo local
Subdomínio / domínio	hermes-local / seudominio.com
Caminho do Access	Vazio: proteger o hostname inteiro
Nome da política	Somente Wabotia
Action / Ação	Service Auth
Include / Incluir	Service Token = wabotia-hermes-local

5. Salve a política e a aplicação. Não use Everyone ou Bypass.

6. Repita para **api-hermes-local.seudominio.com**, com o mesmo token. Na VPS, substitua local por vps nos nomes.

7. Confirme as duas aplicações e suas políticas antes de publicar as rotas do túnel.

Service Auth permite que o Worker se identifique sem uma pessoa no navegador. Uma política de login por e-mail não substitui essa autenticação. A tabela é um mapa dos campos, não uma captura de uma configuração já salva.

Tunnel: crie a ponte

1. Abra **Rede > Túneis > Criar túnel**. Em algumas contas: Zero Trust > Networks > Connectors > Cloudflare Tunnels.

2. Escolha **Cloudflared**, se solicitado. Use um nome novo: wabotia-hermes-local; para VPS, wabotia-hermes-vps.

3. Na tela de instalação, selecione o sistema da máquina do Hermes. No Ubuntu, escolha o pacote/arquitetura corretos e instale pelos comandos oficiais exibidos.

4. Confirme a instalação no terminal:

```
cloudflared --version
```

5. A tela fornece um comando para registrar o conector. Ele contém o **token do túnel**. Guarde esse token privado; não o cole nos campos do Wabotia.

6. Se não houver outro serviço cloudflared, o comando oficial de instalação de serviço pode ser usado. Se já houver, use o serviço separado da próxima página.

7. Aguarde o estado **Healthy / Íntegro**. Depois configure as rotas da página 19.

Referência: o que cada nome significa

Nome	Uso
wabotia-hermes-local	Túnel novo para o computador local.
hermes-local.seudominio.com	Endereço HTTPS para descoberta de perfis.
api-hermes-local.seudominio.com	Endereço HTTPS para a API dos bots.
UUID.cfargotunnel.com	Destino DNS criado pela Cloudflare para o túnel.

Não execute outro **cloudflared service install** por cima de um conector existente. O token do túnel e o par Client ID/Secret do Access são credenciais diferentes. Não abra portas de entrada no roteador ou na VPS.

Um serviço de túnel separado

Ubuntu/Linux/WSL2 com systemd: no Bash da máquina do Hermes, guarde o token sem exibi-lo no histórico. Cole **somente o token** quando solicitado. A entrada fica invisível; pressione Enter.

```
mkdir -p ~/.config/wabotia/tunnel ~/.config/systemd/user
chmod 700 ~/.config/wabotia/tunnel
read -rsp "Token do novo túnel: " WABOTIA_TUNNEL_TOKEN
printf "%s" "$WABOTIA_TUNNEL_TOKEN" > \
  ~/.config/wabotia/tunnel/token
unset WABOTIA_TUNNEL_TOKEN
chmod 600 ~/.config/wabotia/tunnel/token
command -v cloudflared
```

Crie **~/.config/systemd/user/cloudflared-wabotia.service** no editor. Ajuste /usr/bin/cloudflared para o caminho mostrado acima. %h representa a pasta do usuário.

```
[Unit]
Description=Cloudflare Tunnel exclusivo do Wabotia
After=network-online.target

[Service]
ExecStart=/usr/bin/cloudflared tunnel --no-autoupdate run \
  --token-file %h/.config/wabotia/tunnel/token
Restart=always
RestartSec=5
UMask=0077

[Install]
WantedBy=default.target

systemctl --user daemon-reload
systemctl --user enable --now cloudflared-wabotia.service
sudo loginctl enable-linger "$USER"
```

No Mac/Windows nativo, siga o serviço oficial para esse sistema, preservando conectores existentes. Para teste temporário, cloudflared tunnel run --token-file CAMINHO_PRIVADO_DO_TOKEN mantém o túnel enquanto o terminal estiver aberto.

Rotas públicas e DNS

No túnel, abra **Published application routes** (ou Public Hostnames) e adicione duas rotas. Use os mesmos hostnames já protegidos pelo Access. A tabela é referência de preenchimento.

Campo	Administrativo	Bots
Subdomínio	hermes-local	api-hermes-local
Domínio	seudominio.com	seudominio.com
Path no Tunnel	^/api/profiles$	^/p/wabotia/v1/.*$
Tipo	HTTP	HTTP
URL do serviço	127.0.0.1:9119	127.0.0.1:8642
HTTP Host Header	127.0.0.1:9119	Deixar padrão

1. Em **Additional application settings > HTTP Settings**, defina o HTTP Host Header administrativo. Isso permite a validação de Host do backend loopback; Access e Bearer continuam obrigatórios.

2. Salve as duas rotas. Na VPS, use hermes-vps e api-hermes-vps. Se mudou a porta administrativa, ajuste os dois campos da rota.

3. Em **Domínios > seu domínio > DNS > Registros**, confira os CNAMEs criados para UUID.cfargotunnel.com, com proxy ativo. Não substitua registros que já tenham outra função.

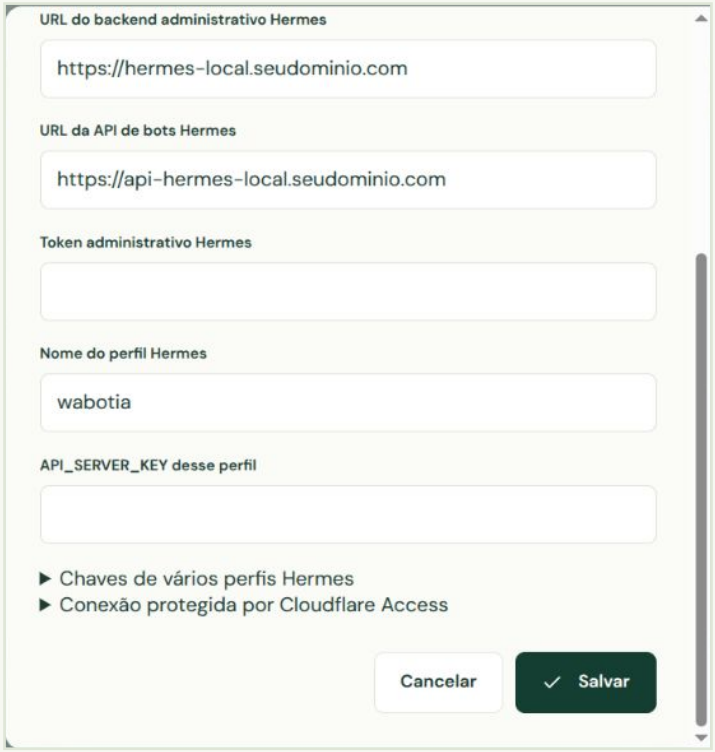
4. Abra uma rota protegida em janela anônima, sem chaves. O Access deve bloquear. Uma tela de login/recusa é esperada; a raiz / pode retornar 404.

5. Se aparecer JSON de perfis sem autenticação, remova a rota publicada e corrija a proteção antes de continuar. O teste autenticado será pelo Wabotia.

Dois campos diferentes: o caminho no Access ficou vazio para proteger o hostname inteiro. O Path do Tunnel limita as rotas publicadas. A API deste exemplo permite apenas o perfil wabotia. Não publique todas as páginas administrativas do Hermes.

Cadastre no Wabotia

Na URL da sua instalação, abra **/app**. Crie um perfil de atendimento e vá a **Conexões > Nova conexão > Hermes**.



Tela de referência do Wabotia, com dados fictícios e chaves vazias.

Campo	Preencher
URL administrativa	https://hermes-local.seudominio.com
URL dos bots	https://api-hermes-local.seudominio.com
Token administrativo	Token administrativo Hermes de chaves.local.txt
Perfil e API_SERVER_KEY	wabotia e sua chave de chaves.local.txt
Cloudflare Access	Abra os detalhes e cole Client ID + Client Secret

Use URLs base, sem /api/profiles ou /p/wabotia/v1. Salve, **Teste a conexão** e **Descubra bots**. Ative a conexão e escolha wabotia como Atendente em Equipe Bot. Mantenha o atendimento pausado até testar os canais.

Se desligou, como religar

Ubuntu nativo, com os serviços deste guia

```
systemctl --user start hermes-gateway.service
systemctl --user start hermes-wabotia-admin.service
systemctl --user start cloudflared-wabotia.service
python3 scripts/prepare-hermes.py --check
```

Se desativou a API na configuração, repita a preparação da página 11 e depois **systemctl --user restart hermes-gateway.service**. O assistente preserva as chaves existentes.

Docker na VPS

```
docker start hermes
sudo systemctl restart hermes-wabotia-admin.service
systemctl --user start cloudflared-wabotia.service
docker exec --user hermes hermes python3 \
  /opt/data/wabotia-connection/prepare-hermes.py \
  --output /opt/data/wabotia-connection --check
```

Se desativou a configuração da API, repita a preparação dentro do contêiner com o mesmo --output e reinicie hermes. Se usou o serviço cloudflared oficial, utilize seu nome correspondente.

Sintoma	Ação
Connection refused	Iniciar gateway/backend e conferir portas.
502 na Cloudflare	Túnel não alcança a API. Execute --check local.
1033 / túnel offline	Ligar cloudflared; conferir token e internet.
401/403 ou login HTML	Conferir chaves, validade e política Service Auth.
Invalid Host	HTTP Host Header = 127.0.0.1:9119.
Perfil não executa	Conferir modelo/provedor e chave do perfil.

Com dois OK locais e túnel Íntegro, volte ao Wabotia e teste a conexão. Computador/WSL precisam ficar ligados. O painel permanece disponível quando o Hermes está offline.

WhatsApp: Meta direto

Escolha este modo para receber a API oficial Meta diretamente no Worker. O Hermes é chamado pelas APIs nativas; não há plugin Wabotia nesse caminho.

1. Na Meta, prepare a aplicação WhatsApp Business e um número de teste. Separe o token, **Phone Number ID**, **WABA ID** e **App Secret**.

2. No Wabotia, abra **Conexões > Nova conexão** e escolha **WhatsApp · API oficial Meta**. Informe o número com DDI, somente dígitos, e os dados da etapa anterior.

3. Crie um **Verify Token** aleatório e preencha o campo. Salve a conexão.

4. Copie o webhook exibido no cartão da conexão. Na aplicação Meta, cadastre essa URL e o mesmo Verify Token. Assine os eventos **messages**.

5. Clique **Testar conexão**, ative o canal e use somente destinatários de teste autorizados na Meta.

6. Teste uma mensagem, a resposta pelo painel e um anexo. Só depois prepare o número de produção.

```
https://SUA_URL/webhooks/meta/ID_DA_CONEXAO
```

Janela, consentimento e templates

Fora da janela de atendimento de 24 horas, esta versão permite notificações de protocolo somente com consentimento e template **UTILITY aprovado**. O corpo deve ter **{{1}}** para protocolo e **{{2}}** para andamento. Configure nome e idioma em Menus e preferências.

> **Um número, um consumidor.** Antes de trocar entre Meta direto, QR ou Hermes, pause o antigo, pare o conector correspondente e confira as pendências em Entregas. Ative o novo modo somente depois da reconciliação.

A confirmação perdida de um envio pode produzir estado unknown. Confira no canal antes de autorizar nova tentativa: ela pode duplicar uma mensagem já recebida.

QR direto ou WhatsApp Hermes

QR direto: processo ao lado do Hermes

1. Crie uma conexão **WhatsApp · QR direto**. Informe o número. Salve e clique **Gerar chave do conector**. Guarde chave e ID da conexão.

2. No host que ficará ligado, abra **connectors/qr**. Copie .env.example para .env e preencha URL HTTPS, ID e chave. Instale e inicie:

```
cd connectors/qr
npm ci
npm start
```

3. Ative a conexão. O QR aparece no cartão do painel. No WhatsApp do celular, abra **Aparelhos conectados** e leia o código.

4. Mantenha o processo ligado. Preserve **data/**, com sessão e diário; configure reinício e backup. Não precisa abrir portas de entrada nem túnel para esse conector.

Em Docker, após preencher .env nessa pasta, use **docker compose up -d --build**. Preserve o volume qr-data ao atualizar. Baileys é um cliente não oficial; o pareamento deve ser validado na sua conta.

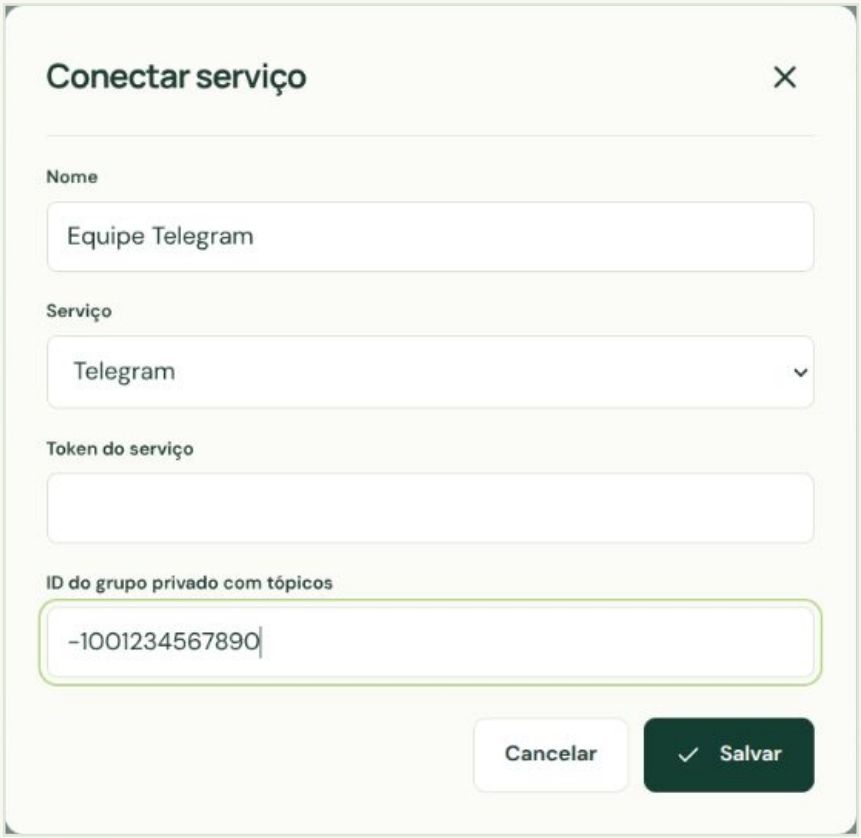
WhatsApp que já está ligado no Hermes

Mantenha Meta/QR configurados no Hermes e escolha **WhatsApp · pelo Hermes** no painel. Esse caminho precisa do transporte mínimo e da extensão compatível de entrada exclusiva no gateway.

Com o gateway parado e o perfil copiado para backup, siga o README de **connectors/hermes**: conferir a revisão, testar o patch com git apply --check, instalar no perfil existente e habilitar wabotia_transport em in_process. Reinicie pelo processo que já utiliza.

Não misture os caminhos. WhatsApp direto e Telegram direto dispensam esse plugin. A extensão do Hermes requer manutenção técnica e revalidação quando o gateway é atualizado; não aplique em outra revisão sem conferir compatibilidade.

Telegram para sua equipe



Nova conexão > Telegram. O grupo é fictício e o token não está preenchido.

1. Crie um bot exclusivo no **@BotFather** com /newbot. Não reutilize o bot do gateway Hermes.

2. Crie um **grupo privado com tópicos**. Adicione o bot como administrador, com permissão de gerenciar tópicos e enviar mensagens e anexos.

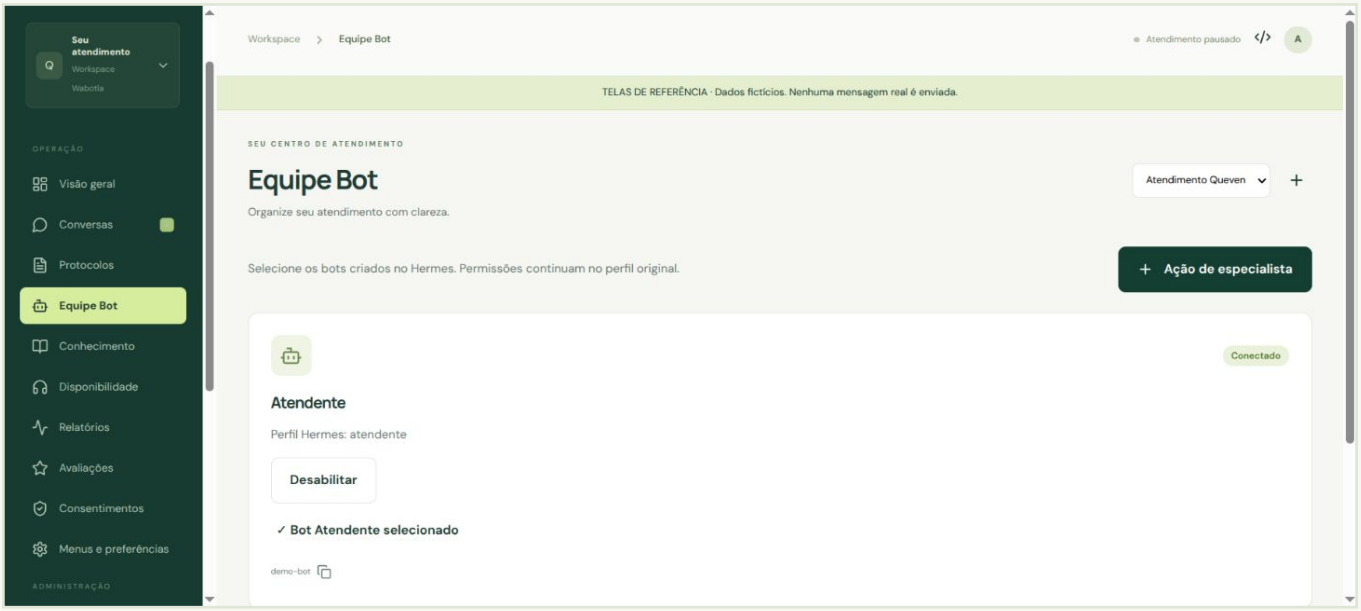
3. Obtenha o ID numérico do grupo, geralmente -100...; o Bot API getUpdates pode mostrar esse ID antes de configurar o webhook.

4. No painel, informe token e grupo, salve, teste e clique **Ligar recebimento Telegram**. Ative a conexão. Em Menus e preferências, cole os IDs da conexão e do grupo.

5. Cada pessoa envia **/identificar** no grupo. Vincule o ID retornado à conta correta em Pessoas. Depois use **/presente**.

O Telegram conversa diretamente com o Worker. Não instala plugin no Hermes. Vincule apenas o ID verificado da pessoa; não use bots terceiros para acessar os dados do grupo.

Prepare os bots e os menus



Equipe Bot com um perfil de demonstração. Os dados são fictícios.

1. Em **Equipe Bot**, habilite os perfis descobertos e selecione quem será o **Atendente**. Só habilite perfis cuja API_SERVER_KEY foi cadastrada.

2. Clique **Ação de especialista**. Escolha o bot, descreva quando consultá-lo e a ação permitida. Os modelos e ferramentas continuam configurados no Hermes.

3. Abra **Conhecimento** e cadastre informações úteis: texto, URL pública, PDF com texto ou DOCX. Espere a importação ficar disponível. Arquivos têm limite de 20 MB.

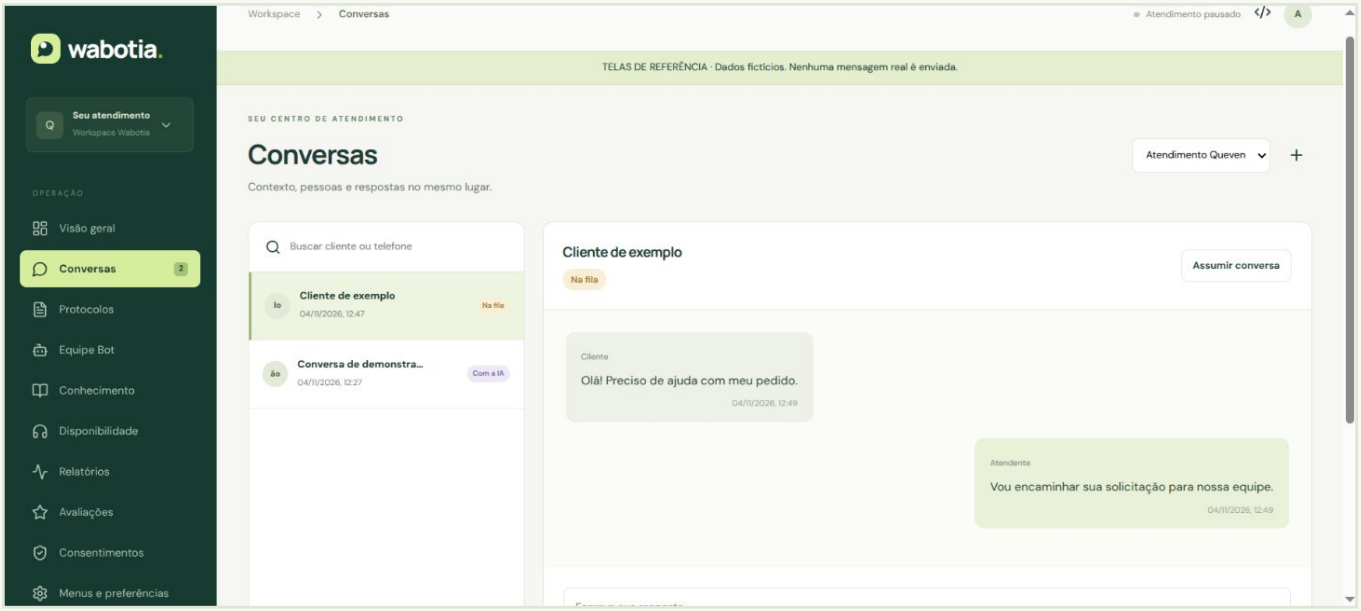
4. Em **Menus e preferências**, configure prazo, instruções, avaliações e opções de menu. Se necessário, use o editor JSON para aplicativos e submenus. JEV é opcional.

5. Cadastre pessoas e permissões por perfil. Em **Disponibilidade**, registre presença antes de abrir o atendimento.

Arquivos de bots: precisam ser exportados como artefatos da API nativa. Caminhos locais não viram links públicos. Downloads de uso único interrompidos exigem nova exportação; o Wabotia não os repete automaticamente.

PDF digitalizado sem texto não tem OCR nesta versão. Vídeos precisam de legendas públicas/autorizadas; também é possível importar TXT, VTT ou SRT.

Faça a primeira conversa



Tela de atendimento com mensagens fictícias. Assumir invalida a resposta automática anterior.

1. Use um número de teste autorizado. Envie uma pergunta que possa ser respondida com o conhecimento cadastrado.

2. Abra a conversa e clique **Assumir**. Confira que o responsável é sua conta e responda pelo painel.

3. Teste nota interna, anexo, transferência e abertura de protocolo. Nota interna não deve ir ao cliente.

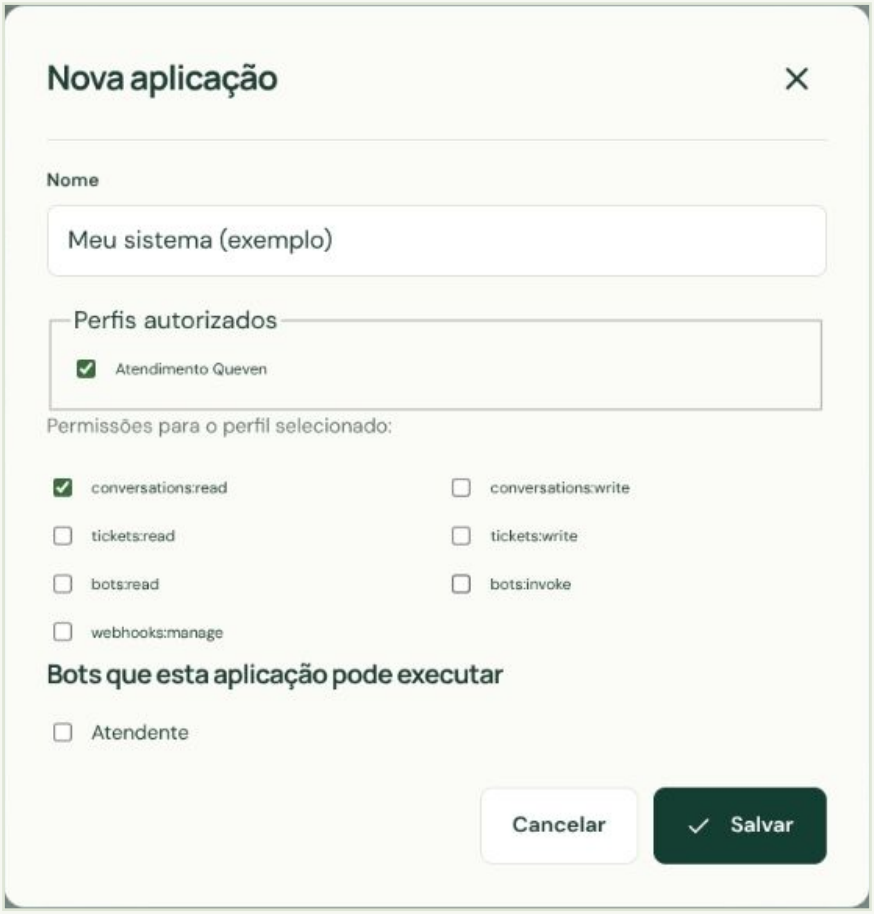
4. No Telegram, vá ao tópico da conversa e use /assumir antes de /responder. Confira o resultado também no painel.

Comando Telegram	Uso
/presente ou /ausente	Registra disponibilidade.
/assumir e /responder texto	Assume a conversa e responde ao cliente.
/nota texto e /transferir ID	Registra nota interna e transfere para atendente autorizado.
/encerrar e /retomar	Conclui ou devolve o atendimento à automação.
/identificar e /ajuda	Mostra sua identidade ou a lista de comandos.

No modo por comando, texto comum vira nota interna. No modo direto, o responsável responde sem /responder conforme as regras do tópico. Confira esse comportamento com mensagens fictícias antes de usar.

Integre outros sistemas

A API é para comunicação entre servidores. Seu programa usa uma chave própria e só opera os perfis, bots e ações autorizados.



API e integrações > Nova aplicação. Exemplo de seleção de permissões.

1. Cadastre uma aplicação, escolha os perfis e marque as permissões necessárias. Selecione os bots que ela poderá executar.

2. Clique **Gerar chave**, escolha validade e copie o valor exibido para o backend do integrador. Para rotação, emita outra e depois revogue a antiga.

3. Abra **/api-docs** na sua instalação. Há OpenAPI 3.1 e exemplos em cURL, JavaScript e Python no repositório.

4. Cadastre um webhook HTTPS para eventos. Guarde o segredo HMAC e verifique assinatura, timestamp e ID do evento. O destinatário precisa deduplicar.

Escritas exigem Idempotency-Key. Bots retornam 202 e um ID para acompanhamento. Repetir a chave com outro corpo gera conflito; timeout não deve disparar uma nova execução automaticamente.

Backup e manutenção

Copiar seus dados

1. Pause o atendimento e as entradas dos canais. Pare QR/transporte para não gravar durante a cópia. Preserve os diários.

2. Na pasta Wabotia, execute o comando. Use o arquivo de configuração da implantação correta.

```
node scripts/backup.mjs wrangler.production.local.json backup-seguro
```

O backup inclui banco e mídias. Só considere concluído quando manifest.json tiver **complete: true**. Guarde a pasta protegida e a **MASTER_KEY original separadamente**. Depois retome os canais.

Restaurar em recursos novos

Crie um D1 vazio e um R2 privado novo. Faça uma configuração local apontando para esses recursos, sem sobrescrever a produção. Então execute:

```
node scripts/restore.mjs wrangler.restore.local.json backup-seguro --empty-target
```

O script recusa banco preenchido. Configure a MASTER_KEY original, publique a mesma versão e teste dados e arquivos antes de mover o domínio. Pare a instalação antiga antes de ligar os mesmos canais na restaurada.

Diagnóstico

```
node scripts/diagnose.mjs https://SUA_URL
```

Sintoma	O que conferir
Hermes offline	Processo, portas, túnel, Access, URLs e chaves de cada perfil.
401 / 403 / 404	Validade da chave, permissões, perfil e dono do recurso.
429	Respeite Retry-After e reduza a frequência.
unknown	Confira Hermes/canal e use Execuções ou Entregas para reconciliar.

Antes de atender clientes

O núcleo e a publicação já foram testados. Seus perfis e canais precisam passar por homologação com credenciais e destinatários autorizados. Não confunda uma tela de demonstração com um canal conectado.

Execute na sua instalação de teste

1. Envie e receba texto e anexo em cada canal escolhido. Confira protocolo, prazo, consentimento, avaliações, presença, notas e transferência.

2. Peça uma resposta lenta ao bot e assuma no meio da execução. A resposta automática antiga não deve chegar ao cliente.

3. Desligue e religue Hermes, túnel e conector. O painel deve continuar acessível; confira a retomada e os estados sem confirmação.

4. Repita uma mensagem/webhook de teste. Confira deduplicação, assinatura e recuperação. Teste também uma chave expirada/revogada.

5. Valide backup e restauração antes de depender da instalação. Só então ative o atendimento para clientes.

Evidência desta edição: 45 testes em workerd, 3 testes do atualizador, 3 testes Python do transporte, 20 verificações HTTP em staging, exemplos em três linguagens e restauração remota D1/FTS/R2. Os testes externos usaram simulações ou dados fictícios. **As APIs Hermes locais também passaram na descoberta e na recusa de acesso sem chave. Isso não substitui homologação completa de modelo, túnel e canais Meta, QR e Telegram na sua instalação.**

Onde encontrar os detalhes

Site e guia atualizado

Código aberto e instruções completas

Matriz de validação e limites

Operação, atualização e recuperação

Uma organização por implantação. Fuso America/Sao_Paulo. O histórico antigo não é importado automaticamente. Telas capturadas em 04/11/2026; menus de fornecedores podem mudar. Wabotia é independente de Cloudflare, Meta, Telegram e NousResearch.

Autorize o deploy do Hermes

Depois de ligar as APIs locais e obter dois OK, você pode criar Access, túnel e DNS pelo Wabotia. O Hermes continua no computador ou VPS. Quem já conectou esta máquina pode manter a conexão existente.

1. Ative Zero Trust e confirme que o domínio está ativo na Cloudflare. As adesões a R2 e aos planos continuam sendo feitas pelo titular.

2. Abra **My Profile > API Tokens > Create Token > Custom Token**. Limite o token à conta e à zona do domínio desejado.

3. Na conta, conceda **Cloudflare Tunnel: Edit**, **Access: Apps and Policies: Edit** e **Access: Service Tokens: Edit**. Na zona, conceda **Zone: Read** e **DNS: Edit**. Defina validade e copie o token em local privado.

4. Na visão geral do domínio, copie **Account ID** e **Zone ID**.

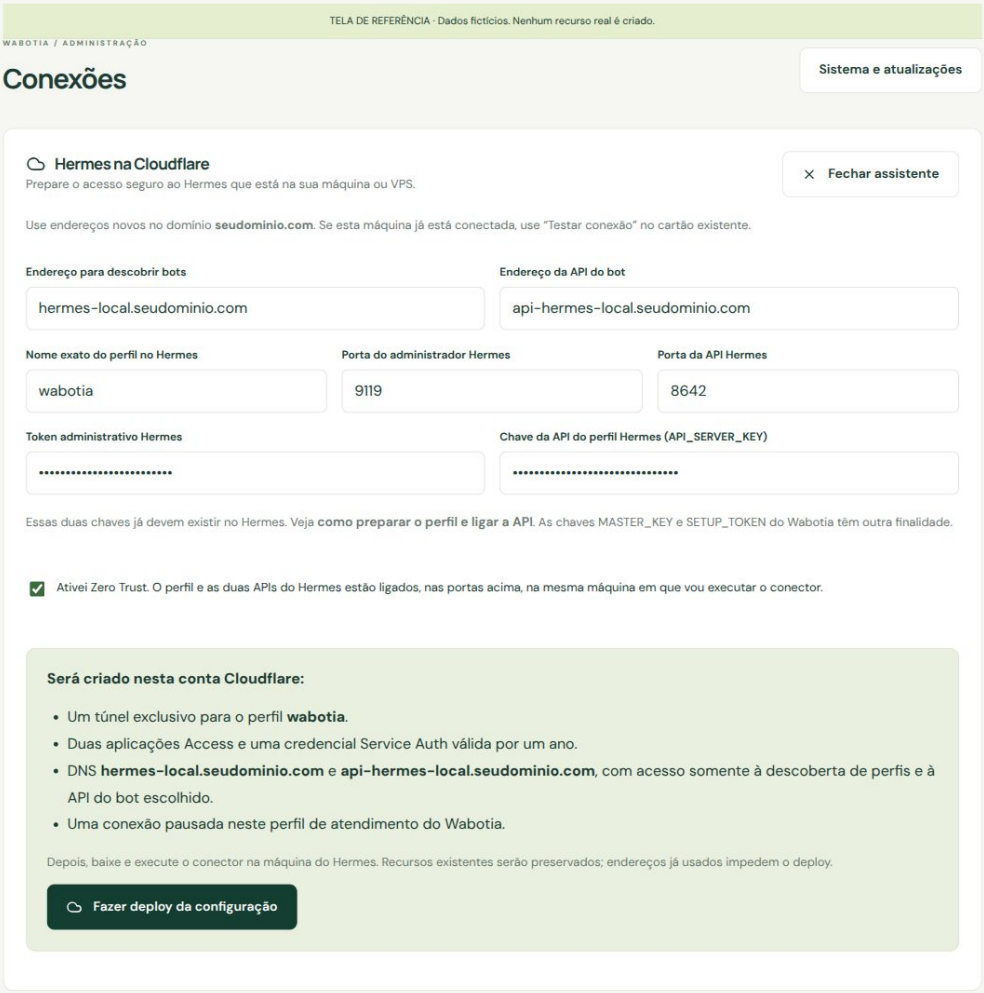
5. No Wabotia, abra **Sistema e atualizações > Preparação do Hermes na Cloudflare**. Cole os dois IDs e o token. Clique **Verificar e salvar autorização**.

Qual chave é esta? É o token de API da Cloudflare, usado para criar os recursos. Ele é diferente de MASTER_KEY/SETUP_TOKEN, das duas chaves Hermes e do par Client ID/Secret do Access. O Wabotia o guarda criptografado.

Se aparecer erro de autorização, confira conta, zona, validade e todas as permissões. Um token somente de leitura não pode executar o deploy.

Roteiro completo do deploy pelo painel

Revise e faça o deploy



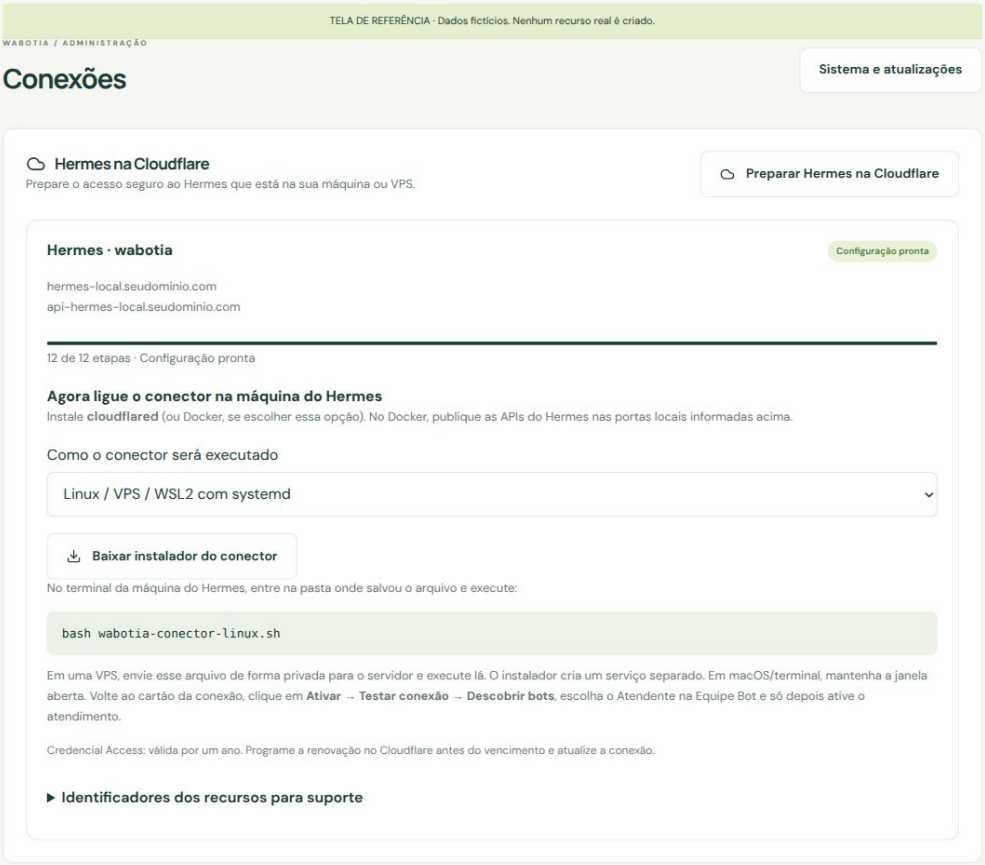
Conexões > Preparar Hermes na Cloudflare. Tela funcional com dados e chaves fictícios.

1. Informe dois subdomínios novos, perfil Hermes, portas e as duas chaves de chaves.local.txt. Confirme os pré-requisitos e clique **Revisar configuração**.

2. Confira o resumo e clique **Fazer deploy da configuração**. Acompanhe as 12 etapas. Endereços usados serão preservados e impedirão a criação.

A credencial Access gerada vale por um ano. Renove-a no Cloudflare e atualize a conexão antes de vencer. Não execute o assistente nos endereços de uma conexão já pronta.

Ligue o conector da máquina



Ao concluir o deploy, baixe o conector para a máquina do Hermes. Exemplo fictício.

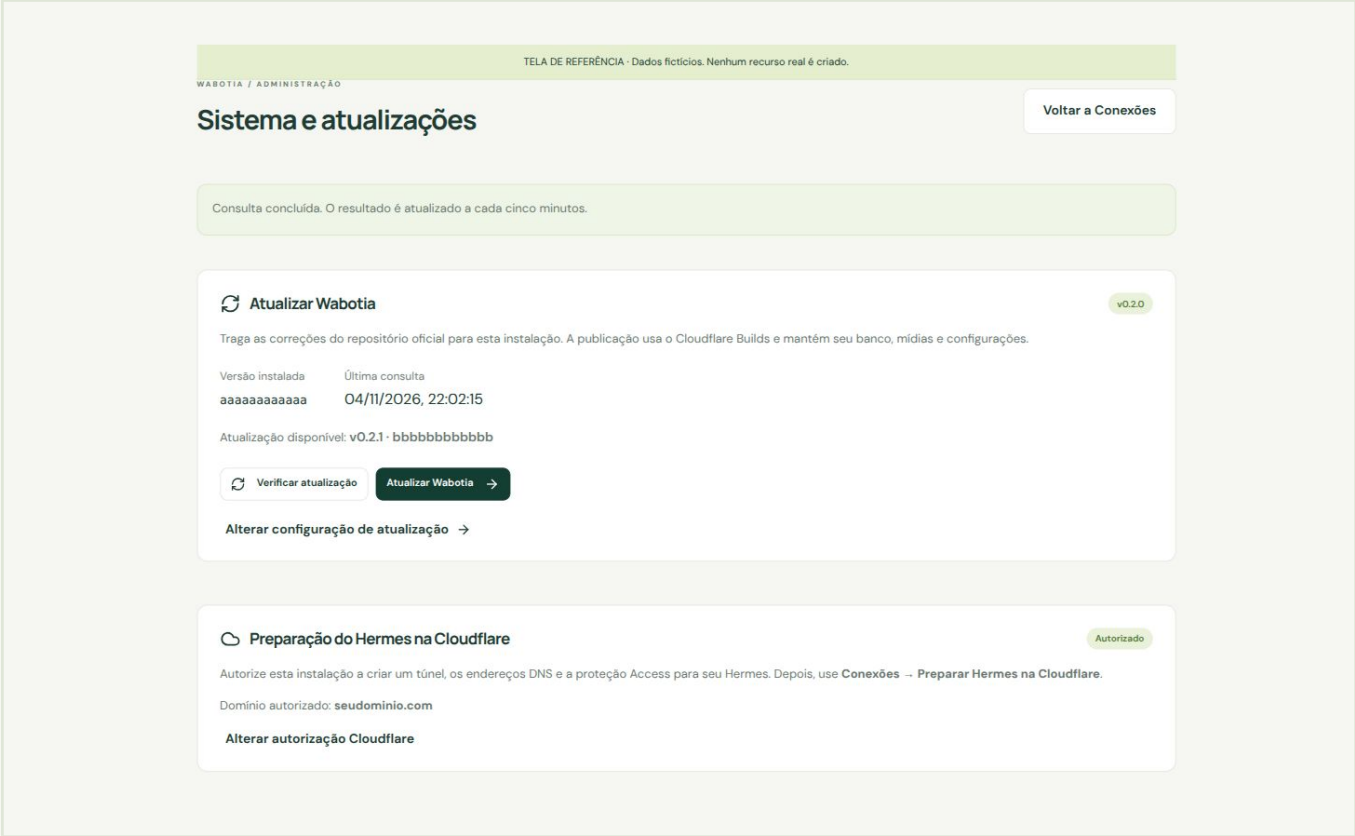
1. Instale cloudflared pelo site oficial, ou Docker para essa opção. Escolha o sistema e baixe o instalador. O arquivo contém uma credencial: guarde-o em local privado.

2. No terminal da máquina do Hermes, entre na pasta do download e rode o comando correspondente. Na VPS, transfira o arquivo de forma privada antes.

```
bash wabotia-conector-linux.sh
```

Linux/WSL2 precisa de systemd. Docker requer Linux com rede do host. macOS/terminal precisa permanecer aberto. Na VPS, habilite sudo loginctl enable-linger "$USER". Depois, ative a conexão no painel, teste, descubra os bots e selecione o Atendente.

Atualize pelo painel



Sistema e atualizações. Referência com uma atualização fictícia disponível.

1. Após a configuração inicial da próxima página, clique **Verificar atualização**. A consulta ao repositório é renovada a cada cinco minutos.

2. Se houver uma versão disponível, clique **Atualizar Wabotia**. A Cloudflare prepara e publica o código; aguarde alguns minutos.

3. Só considere concluído quando aparecer **Atualização publicada e verificada**. Clique **Reabrir painel atualizado** para carregar as novas telas.

Pedido sem confirmação? Confira Deployments/Builds do Worker. O pedido pode ter sido aceito antes de a conexão cair. Não repita automaticamente. Após falha confirmada ou 15 minutos, o painel permite liberar nova tentativa, exigindo confirmar que não há build em execução ou fila.

Prepare a atualização uma vez

1. Faça backup. Na Cloudflare, abra **seu Worker > Settings > Builds**. Confira o repositório da instalação e a branch main.

2. Confirme que existe **wabotia-build.mjs** na raiz desse repositório. Se sua cópia é antiga, siga o roteiro no link abaixo antes de continuar.

3. Em Build configuration, informe os comandos abaixo.

```
Build command: node wabotia-build.mjs
Deploy command: npm run deploy
```

4. Em **Variables and secrets de Builds**, adicione WABOTIA_UPDATE_ORIGIN com o endereço HTTPS da instalação, sem /app no final.

5. Em **Deploy Hooks**, crie um hook para main e copie sua URL. No Wabotia, abra **Sistema e atualizações > Configurar atualização pelo painel**, cole no campo protegido, marque a confirmação e salve.

```
WABOTIA_UPDATE_ORIGIN
https://meu-wabotia.minhaconta.workers.dev
```

A URL do hook é uma credencial. Não publique em GitHub nem screenshots. Builds é separado de GitHub Actions, que continua desativado. A publicação preserva banco, mídias, domínios, variáveis e secrets. Novos recursos ou mudanças incompatíveis exigem revisão manual.

Cópia antiga sem o menu Sistema?

Baixe wabotia-build.mjs de um commit testado do repositório oficial e adicione somente esse arquivo à raiz da cópia da instalação. Configure os comandos, a origem e, temporariamente, WABOTIA_BOOTSTRAP_COMMIT com o SHA completo de 40 caracteres. Inicie o build pelo dashboard. Verifique /api/health e o painel, remova a variável temporária e configure o hook. Preserve wrangler.jsonc e os secrets originais.

Primeira atualização e recuperação, passo a passo

Os testes automáticos simulam as APIs de criação Cloudflare. Criar uma nova conexão real pelo assistente ainda precisa ser validado na conta de destino, com seu token autorizado.

Se a consulta de versões falhar

Use esta recuperação apenas quando uma versão antiga não consegue verificar atualizações. Ela publica uma correção oficial pelos Builds e preserva os recursos da sua instalação.

1. Faça backup e confira que não há compilação em execução ou na fila da Cloudflare.

2. No repositório oficial, abra **Commits**, escolha a correção revisada e copie seu **SHA completo**: o código de 40 caracteres que identifica essa versão.

3. Baixe **wabotia-build.mjs** desse commit. No GitHub da sua instalação, use **Add file > Upload files** para substituir somente esse arquivo e confirme o commit. Aguarde qualquer compilação automática terminar.

4. Na Cloudflare, abra **seu Worker > Configurações > Compilações**. Preserve WABOTIA_UPDATE_ORIGIN. Troque temporariamente o comando de build pelo comando abaixo, substituindo SHA_COMPLETO pelo código copiado. Mantenha npm run deploy como comando de implantação e salve.

```
node wabotia-build.mjs --commit SHA_COMPLETO
```

5. Em **Implantações**, abra a última compilação e clique **Retry build / Repetir compilação**. Aguarde a conclusão.

6. Abra o endereço da instalação seguido de **/api/health**. Confira a versão e o commit. Reabra /app e confira suas conexões.

7. Volte às configurações de Compilações, retire --commit e o código, deixando somente o comando abaixo, e salve.

```
node wabotia-build.mjs
```

Não pule o último passo. Se deixar o código fixo no comando, as próximas publicações continuarão nessa versão. Depois de restaurar o comando, use Verificar atualização e Atualizar Wabotia normalmente.

Roteiro completo de recuperação